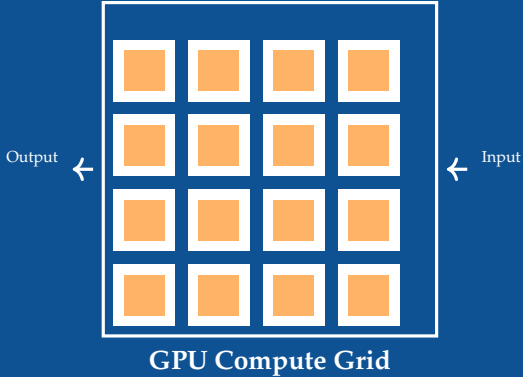


AI INFERENCE ENGINEERING

Building Fast, Scalable, and Production-Ready AI Systems

Chapter 1

Introduction to AI Inference Engineering



First Edition · 2025

CHAPTER 1

Introduction to AI Inference Engineering

“Any sufficiently advanced technology is indistinguishable from magic. Our job as engineers is to make that magic fast, cheap, and reliable.”

— Arthur C. Clarke (adapted)

Learning Objectives

1. Understand the precise meaning of **inference** and how it differs from model training.
2. Explain why inference engineering has become one of the most important disciplines in modern AI infrastructure.
3. Describe the full lifecycle of a request through a production inference system — from API call to token response.

4. Understand the key performance metrics used to evaluate inference systems: **latency**, **throughput**, and **cost per token**.
5. Identify the main hardware components (GPU, CPU, memory, network) and their roles in inference.
6. Trace the historical evolution from research models to today's hyper-scale inference platforms.
7. Recognise the real-world systems — ChatGPT, Gemini, Claude, Mid-journey — that rely on the engineering concepts taught in this book.

1.1 Why This Book Exists

Every time you send a message to ChatGPT, ask Gemini to summarise a document, or watch Midjourney paint an image from your words, something remarkable is happening behind the scenes.

Thousands of specialised computers — each containing powerful graphics processors, hundreds of gigabytes of memory, and high-speed networking — are working together at millisecond timescales to run an AI model and deliver your result.

That entire operation is called **AI inference**. The engineering discipline that designs, builds, optimises, and scales these systems is called **AI inference engineering**.

This book will teach you that discipline — from the ground up.

Who This Book Is For

This book is written for:

- Software engineers who want to move into AI infrastructure
- ML engineers who want to understand what happens after model training
- Platform engineers building internal AI serving systems

- Technical leads making architecture decisions about AI deployments
 - Anyone curious about how large AI systems actually work at scale
- You do **not** need a PhD. You do need curiosity and a willingness to think carefully about systems.

1.2 What Is Inference? A First-Principles Explanation

1.2.1 Start with the Simplest Idea

Let's start with something familiar.

Imagine you study very hard for a maths exam. You spend weeks reading textbooks, working through problems, and learning formulas. That studying phase — where you absorb knowledge — is the *learning* phase.

Then exam day arrives. Someone gives you a new problem you have never seen before. You apply everything you have learned to produce an answer. That answering phase is *inference*.

★ The Exam Analogy

Training = studying for months to learn how to solve problems

Inference = sitting the exam and solving a new problem using what you learned

The student does not re-read all their textbooks during the exam. They use their *trained knowledge* — stored in their brain — to answer quickly.

AI systems work the same way.

1.2.2 Translating the Analogy to AI

An AI language model like GPT-4 or Claude is trained on hundreds of billions of text documents. During training, the model adjusts billions of numerical

parameters — think of these as the “knowledge” stored in the model — to learn patterns in language.

This training process takes *months*, thousands of GPUs, and costs tens of millions of dollars.

Once training is complete, the model’s parameters are frozen. The model is then *deployed* — made available to users.

When you send a message to ChatGPT:

1. Your message arrives at a server.
2. The server loads the trained model parameters.
3. The model performs a mathematical computation on your message using those parameters.
4. The model outputs a response — word by word, or technically, *token by token*.
5. The response is sent back to you.

That entire process — steps 1 through 5 — is called a single **inference request**.

AI Inference

AI Inference is the process of running a trained AI model on new input data to produce a prediction or output.

The model parameters (weights) do not change during inference. The model simply *applies* its learned knowledge to the input.

1.2.3 The Key Mathematical Operation

If you want to understand inference at its heart, here is the core mathematical reality:

An AI language model is, at its most fundamental level, a very large mathematical function:

$$\text{output} = f(\text{input}, \theta)$$

where:

- **input** is the text you provided (encoded as numbers)
- θ (theta) represents the model's *billions of parameters* — its learned knowledge
- f is the series of mathematical operations the model performs (matrix multiplications, attention computations, activations)
- **output** is the probability distribution over the next token to generate

During inference, θ is **fixed**. Only **input** varies. The model's job is simply to evaluate this function quickly and correctly.

The Core Challenge

GPT-4 has approximately **1.76 trillion parameters**. Each parameter is a floating-point number (typically 16 bits = 2 bytes). That alone is **3.5 terabytes of data** — just to store the model.

Loading, managing, and computing with that much data, at high speed, for millions of users simultaneously, is the fundamental engineering challenge of AI inference.

1.3 Training vs. Inference: Understanding the Difference

This is one of the most important distinctions in AI engineering. Training and inference look superficially similar — both run the model on data — but they are *fundamentally different engineering problems*.

1.3.1 What Happens During Training?

During training, the goal is to *improve* the model's parameters.

The process works like this:

1. Feed a batch of training examples into the model.
2. The model makes predictions (a *forward pass*).
3. Compare the predictions to the correct answers. Compute a *loss* (how wrong was the model?).

4. Use calculus (*backpropagation*) to compute how each parameter contributed to the error.
5. Adjust the parameters slightly in the direction that reduces the error (*gradient descent*).
6. Repeat billions of times.

Step 4 — backpropagation — requires storing intermediate computations from the forward pass so gradients can flow backwards. This is called the **activation memory** or **gradient memory** and it is *enormous* — often 5–10x the size of the model itself.

1.3.2 What Happens During Inference?

During inference:

1. Feed a user’s input into the model.
2. The model makes a prediction (forward pass only).
3. Return the prediction.

That’s it. No backpropagation. No gradient storage. No parameter updates.

1.3.3 Why This Distinction Matters Enormously

The difference between training and inference isn’t just academic — it completely changes the engineering requirements:

Dimension	Training	Inference
Goal	Improve model weights	Apply model to new inputs
Frequency	Once (or periodic)	Millions of times per day
Latency need	Hours/days acceptable	Milliseconds required
Memory use	Very high (gradients)	Lower (weights only)
Compute pattern	Batch, parallel	Often sequential (autoregressive)
Cost driver	GPU-hours of compute	Cost per token at scale

Dimension	Training	Inference
Batching	Large fixed batches	Dynamic, variable batch sizes
Parameter state	Changing constantly	Completely frozen
Primary metric	Loss convergence	Latency / throughput
Engineering focus	Distributed training	Serving optimisation

Table 1.1: Key differences between training and inference workloads.

The Business Reality

OpenAI reportedly spends approximately \$700,000 per day running ChatGPT inference. Google runs billions of AI inference calls daily across its products. Amazon, Microsoft, and Meta each operate inference clusters of tens of thousands of GPUs.

Training is a one-time (or infrequent) cost. **Inference is an ongoing, relentless operational cost.** This is why inference engineering has become a separate discipline.

1.4 Why Inference Engineering Matters

You might wonder: “If inference is just running the model, how complex can it really be?”

The answer is: extraordinarily complex. Let us walk through why.

1.4.1 The Scale Problem

Consider ChatGPT during peak usage:

- Approximately **10 million daily active users**
- Each user sends multiple messages per session
- Each message might generate 200–500 output tokens

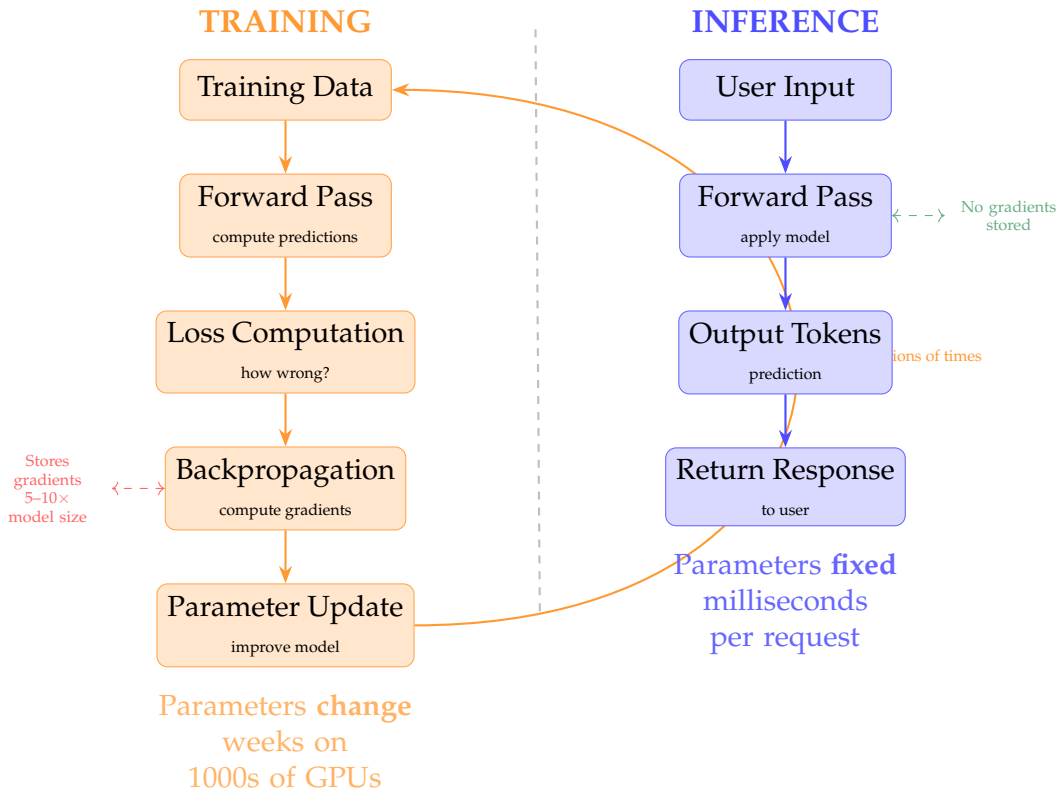


Figure 1.1: Training vs. Inference. Training requires storing large intermediate activations for backpropagation. Inference only needs a single forward pass, making memory requirements dramatically lower.

- Each token requires evaluating a model with hundreds of billions of parameters
- Total: potentially **billions of model evaluations per day**

Now add this constraint: users expect responses within **1–2 seconds**.

This is roughly equivalent to a restaurant that must:

- Serve *millions of customers simultaneously*
- Each meal is a custom five-course dinner (not fast food)
- Every meal must be ready within 2 minutes of ordering
- The kitchen has a fixed, enormous cost to operate

No traditional software system faces this combination of constraints. Inference engineering exists to solve them.

1.4.2 The Three Core Tensions

Every inference engineering decision involves balancing three competing objectives:

- **Latency**: How long does a single user wait for their response? (measured in milliseconds)
- **Throughput**: How many total requests can the system handle per second across all users?
- **Cost**: How much does it cost to serve each request? (measured in dollars per million tokens, often written \$/MTok)

These three objectives *pull against each other*. Making one better often makes another worse.

★ The Bus vs. Taxi Analogy

A **taxi** picks you up immediately (low latency) but can only carry one passenger (low throughput) and is expensive per person (high cost).

A **bus** waits until it is full (higher latency) but carries many passengers at once (high throughput) and is cheap per person (low cost).

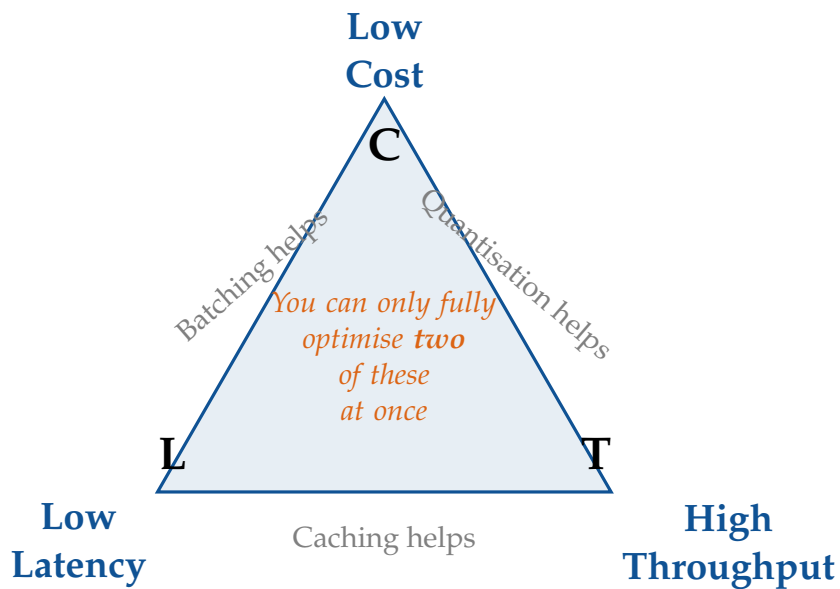


Figure 1.2: The Inference Engineering Trilemma. Every system design must explicitly choose which objectives to prioritise.

Inference systems face exactly this tradeoff. *Batching* is the bus strategy. *Streaming* is the taxi strategy. The best inference engines choose intelligently based on load.

1.4.3 Performance Numbers That Matter in Production

Here are the performance targets that production AI companies actually care about:

Metric	Target (Consumer)	Target (Enterprise)
Time to first token (TTFT)	< 500ms	< 200ms
Tokens per second (TPS)	20–50 tok/s	50–200 tok/s
P99 latency	< 5s	< 2s
Cost per million tokens	\$0.50–\$5.00	\$0.10–\$1.00
GPU utilisation	> 60%	> 80%
Requests per GPU per hour	100–500	500–5000

Table 1.2: Typical production performance targets for LLM inference systems.

What TTFT Means

Time to First Token (TTFT) is the time from when you press “send” to when you see the *first word* of the response appear. It is arguably the most important user-facing latency metric because it determines whether the system feels “responsive” or “slow.”

A system generating 50 tokens per second but with a 3-second TTFT will feel slow, even if the total generation time is short.

1.5 The Anatomy of an Inference System

Before we go deeper, let us trace exactly what happens when you send a message to a production AI system. Understanding this flow is essential — every optimisation technique we will learn in later chapters targets a specific step in this journey.

1.5.1 The Complete Request Lifecycle

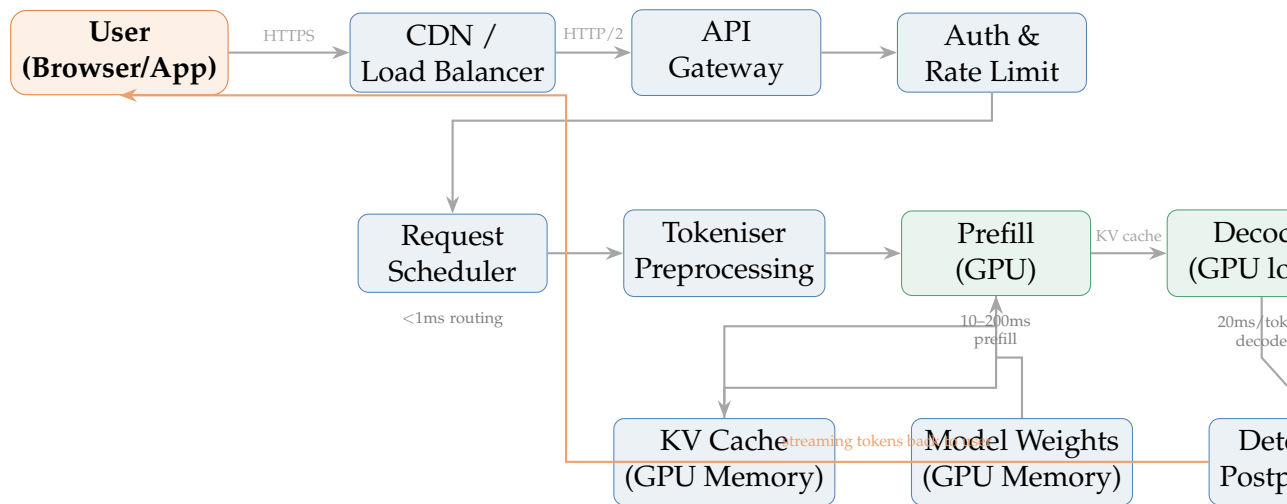


Figure 1.3: Anatomy of a production inference request. Each box represents a system component that we will study in detail in later chapters.

Let's walk through each stage:

Stage 1: Network Ingress (0–5 ms)

Your request travels over the internet to the nearest **CDN edge node** (Content Delivery Network). The request is routed to the closest data centre to reduce network round-trip time.

Stage 2: API Gateway and Authentication (1–10 ms)

The API gateway validates your API key, checks your rate limits, and decides whether to accept or queue your request. This protects the system from being overwhelmed.

Stage 3: Request Scheduling (<1 ms)

The **request scheduler** — one of the most important components in a modern inference engine — decides:

- Which GPU cluster will handle this request?
- Should this request be batched with other pending requests?
- How much priority should this request receive?

Stage 4: Tokenisation (0.5–2 ms)

Your text is converted into **tokens** — numerical IDs from the model’s vocabulary. The word “inference” might become token 23847. “engineering” might become tokens 45231 + 9182 (split into sub-words). This tokenised input is what the GPU will actually process.

Stage 5: Prefill Phase (10–500 ms)

This is where real GPU computation begins. The model processes your *entire input prompt* in parallel to build an internal representation of what you have said.

Think of it as the model “reading and comprehending” your entire message at once before it starts responding.

This phase is compute-intensive and its duration scales with the length of your prompt.

Stage 6: Decode Phase (20–50 ms per token)

Now the model begins generating tokens *one at a time*. Each token requires:

1. Loading the model weights from GPU memory

2. Performing a forward pass through the model
3. Sampling the next token from the probability distribution
4. Appending the new token to the context
5. Repeating for the next token

This sequential nature is what makes LLM inference fundamentally different from other AI workloads — and is the source of many of the key optimisations we will study.

Stage 7: Detokenisation and Return

Each generated token ID is converted back to text and streamed back to you — which is why you see text appearing word by word on your screen rather than all at once.

The Decode Phase Bottleneck

The decode phase is typically the **primary bottleneck** in LLM inference. Why? Because each token generation requires reading the *entire model* from GPU memory — often hundreds of gigabytes. Modern GPUs can read memory at 3–8 TB/s, but that is still the limiting factor. This is called being **memory-bandwidth-bound**, and we will explore it extensively in Chapter 3.

1.6 The Hardware Landscape

AI inference runs on specialised hardware. Understanding this hardware — at least conceptually — is essential for understanding why inference systems are designed the way they are.

1.6.1 The GPU: Heart of Inference

A **GPU** (Graphics Processing Unit) was originally designed to render video game graphics. It turns out that the mathematical operations needed for graph-

ics — thousands of small matrix multiplications in parallel — are identical to the operations needed for AI neural network inference.

★ CPU vs. GPU: The Factory Analogy

A CPU is like a small team of highly-skilled engineers. There are 8–64 of them. Each one is incredibly smart, can handle complex tasks, and can switch between very different jobs instantly. But you only have a few of them.

A GPU is like a factory floor with 10,000 workers. Each individual worker is less capable — they can only do simple arithmetic. But having 10,000 of them working simultaneously means they can process enormous amounts of data in parallel.

Neural network inference is essentially: “multiply these 100 billion numbers together in the right way.” That’s a perfect job for 10,000 parallel workers.

1.6.2 Key GPU Specifications That Matter

When you hear about an NVIDIA H100 or A100 GPU being used for AI inference, here are the numbers that actually matter:

Specification	NVIDIA (80G)	A100	NVIDIA (80G)	H100	Why It Matters
VRAM (memory)	80 GB		80 GB		Model must fit in VRAM
Memory band-width	2.0 TB/s		3.35 TB/s		Decode speed
FP16 TFLOPS	312		989		Prefill speed
Tensor cores	432		528		Matrix multiply speed
NVLink band-width	600 GB/s		900 GB/s		Multi-GPU communication

Specification	NVIDIA (80G)	A100	NVIDIA (80G)	H100	Why It Matters
TDP (power)	400W		700W		Operational cost

Table 1.3: Key GPU specifications relevant to AI inference.

1.6.3 The Memory Hierarchy

Understanding where data lives during inference is crucial. The memory hierarchy is a pyramid: faster memory at the top but smaller capacity, slower memory at the bottom but larger capacity.

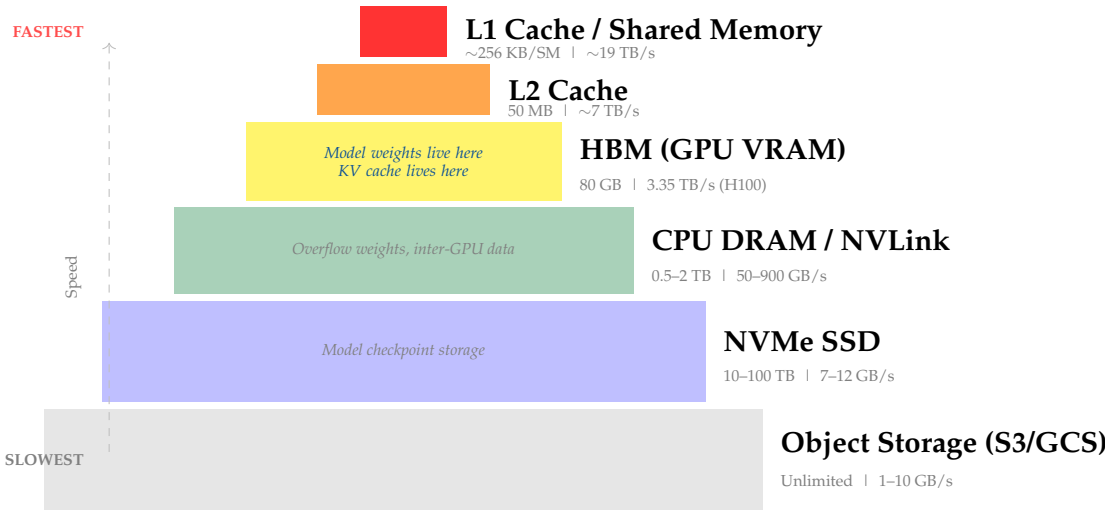


Figure 1.4: The memory hierarchy in a modern GPU inference system. The goal of inference optimisation is to keep hot data as high up this pyramid as possible.

The fundamental rule of inference engineering is: **data that is accessed frequently should live as close to the GPU compute units as possible.**

Every optimisation technique we study — KV caching, quantisation, weight

sharing — can be understood as trying to keep the right data at the right level of this memory hierarchy.

1.7 The Evolution of AI Inference Infrastructure

To understand where we are today and where we are going, we need to understand where we came from.

1.7.1 Era 1: Research-Scale Models (2012–2017)

The deep learning revolution began with AlexNet winning the ImageNet competition in 2012. Models were relatively small (millions of parameters), ran on single GPUs, and inference was simple: run the model, get the prediction, done.

The engineering challenge was primarily training efficiency. Inference was an afterthought.

1.7.2 Era 2: The Rise of Pre-trained Models (2018–2020)

BERT (2018) and GPT-2 (2019) introduced the concept of large pre-trained transformer models fine-tuned for downstream tasks. Models grew to billions of parameters.

Suddenly, inference required serious thought:

- Models no longer fit easily on single GPUs
- Serving latency became a customer-facing concern
- Companies began building dedicated serving infrastructure

TensorFlow Serving and TorchServe emerged as early inference servers.

1.7.3 Era 3: The LLM Explosion (2020–2022)

GPT-3 (2020) with 175 billion parameters changed everything. No single consumer GPU could hold it. Running inference required *model parallelism* across multiple GPUs.

OpenAI, Google, and DeepMind began building specialised inference infrastructure at massive scale. The **transformer** architecture became universal. New optimisation techniques emerged:

- Quantisation (reduce precision to save memory)
- KV caching (avoid redundant computation)
- Tensor parallelism (split the model across GPUs)
- Continuous batching (process requests without gaps)

1.7.4 Era 4: The Commercial Deployment Era (2022–Present)

ChatGPT’s launch in November 2022 caused a step-change in the scale of inference systems. Suddenly, systems that researchers had designed for thousands of queries per day needed to handle *millions*.

This era saw:

- Purpose-built inference engines: **vLLM**, **TensorRT-LLM**, **Triton Inference Server**, **TGI (Text Generation Inference)**
- New serving paradigms: **PagedAttention**, **speculative decoding**, **disaggregated prefill**
- Hardware specialisation: TPUs, AWS Trainium/Inferentia, custom inference chips
- Multi-model serving, router architectures, mixture-of-experts

1.8 Modern Inference Systems: A Tour

Let us briefly survey the major inference engines and systems that power today’s AI products. We will study each of these in detail in later chapters.

1.8.1 vLLM: PagedAttention and Continuous Batching

vLLM, developed at UC Berkeley in 2023, is arguably the most influential open-source LLM inference engine. Its two key innovations are:

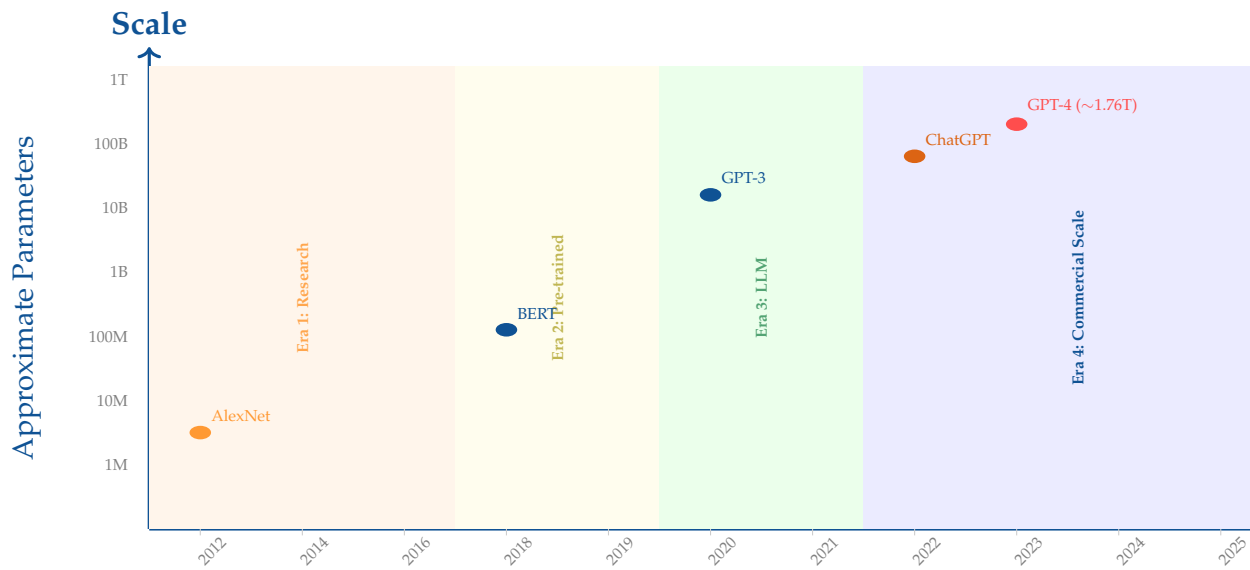


Figure 1.5: Evolution of AI model scale over time, showing the four eras of AI inference infrastructure. Note the logarithmic scale on the y-axis — each step represents a $10\times$ increase in parameters.

- **PagedAttention**: A memory management technique that treats GPU memory like virtual memory in operating systems, reducing memory waste from 60–80% down to under 4%.
- **Continuous Batching**: Instead of waiting for a full batch before processing, vLLM dynamically adds and removes requests from the processing queue, dramatically improving GPU utilisation.

In benchmarks, vLLM achieves **2–4× higher throughput** compared to naive serving approaches.

1.8.2 TensorRT-LLM: NVIDIA’s Production Engine

TensorRT-LLM is NVIDIA’s production inference engine. It compiles model graphs into highly optimised machine code specific to NVIDIA GPU architectures.

Key features:

- Kernel fusion (combines multiple operations into one GPU kernel)
- INT8/FP8 quantisation with hardware-native support
- In-flight batching (similar to continuous batching)
- Custom CUDA kernels for attention, LayerNorm, etc.

TensorRT-LLM typically achieves **40–60% higher throughput** than PyTorch-based serving.

1.8.3 Text Generation Inference (TGI)

Hugging Face’s TGI is widely used for open-source model deployment. It supports a wide range of models and provides production-grade features like tensor parallelism, quantisation, and Prometheus metrics.

1.8.4 Triton Inference Server

NVIDIA’s Triton is a *model-agnostic* serving framework that supports TensorRT, PyTorch, TensorFlow, ONNX, and custom backends. It is commonly used in

enterprise deployments where multiple model types need to be served from a unified platform.

1.8.5 Production Cloud Services

At the cloud provider level:

- **AWS Bedrock:** Serves Claude, Llama, Mistral, and others on Inferentia2 and GPU instances
- **Google Vertex AI:** Serves Gemini on custom TPU infrastructure
- **Azure OpenAI:** Serves GPT-4 on custom NVIDIA clusters
- **Groq:** Uses custom LPU (Language Processing Unit) hardware designed specifically for LLM inference, achieving extremely high token generation speeds (> 500 tokens/second)

1.9 Real-World Applications and Use Cases

Let us ground these concepts by looking at how real systems use AI inference.

1.9.1 Conversational AI (ChatGPT, Claude, Gemini)

The canonical inference use case. Key characteristics:

- Interactive: users expect sub-second TTFT
- Variable length: inputs range from 10 to 100,000+ tokens
- Stateless per request (context is re-sent each time)
- Requires streaming output (word by word display)
- High concurrency: millions of simultaneous users

1.9.2 Code Generation (GitHub Copilot, Cursor)

Code completion requires:

- Extremely low latency ($< 100\text{ms}$) — must feel instant as you type
- High throughput — millions of suggestions per minute
- Short outputs (typically 1–10 lines)
- Context window efficiency (large code files as context)

GitHub Copilot reportedly handles over **1 million suggestions per day** at sub-100ms latency.

1.9.3 Document Processing (RAG, Summarisation)

Retrieval-Augmented Generation (RAG) pipelines combine AI inference with database retrieval:

1. User query $\rightarrow$ embedding model inference (fast, $< 10\text{ms}$)
2. Embedding $\rightarrow$ vector database search
3. Retrieved documents + query $\rightarrow$ LLM inference (slower, 1–10s)

This requires *two* types of inference: embedding models (small, fast, batch-friendly) and generation models (large, sequential).

1.9.4 Image Generation (Stable Diffusion, Midjourney, DALL-E)

Diffusion models have unique inference characteristics:

- Inference requires 20–50 *denoising steps*
- Each step is a full forward pass through a large model
- Output is a 2D image, not text tokens
- Parallelism is easier (image patches can be processed in parallel)
- Users tolerate 5–30 second wait times

1.9.5 Real-Time Applications (Voice, AR/VR)

The frontier of inference engineering:

- Voice assistants require end-to-end latency $< 300\text{ms}$ (speech-to-text + LLM + text-to-speech pipeline)
- AR/VR scene understanding needs 60+ FPS inference
- These require on-device inference (edge deployment)

1.10 Your First Inference System: A Practical Example

Let us make these concepts concrete with a simple but complete Python example. We will serve a language model and measure the key performance metrics.

```
1  """
2  Chapter 1: Basic LLM Inference Example
3  -----
4  This script demonstrates the fundamentals of LLM
   inference:
5  - Loading a model
6  - Running a forward pass (inference)
7  - Measuring latency and throughput
8  - Understanding time-to-first-token vs total
   generation time
9
10 Run with: python basic_inference.py
11 Requires: pip install transformers torch
12 """
13
14 import time
15 import torch
```

```
16 from transformers import AutoModelForCausalLM,
    AutoTokenizer
17
18 # -- Step 1: Load the model and tokeniser
    -----
19 # The tokeniser converts text to token IDs (numbers the
    model understands)
20 # The model contains all the trained parameters
    (weights)
21 #
22 # For this example we use a small model to avoid
    needing large GPU memory.
23 # In production, you would use GPT-4, LLaMA-3, Mistral,
    etc.
24
25 MODEL_NAME = "gpt2" # Small model for illustration
    (~117M parameters)
26
27 print(f"Loading model: {MODEL_NAME}")
28 tokeniser = AutoTokenizer.from_pretrained(MODEL_NAME)
29 model = AutoModelForCausalLM.from_pretrained(
30     MODEL_NAME,
31     torch_dtype=torch.float16, # Use 16-bit precision
        (half memory of float32)
32     device_map="auto", # Automatically place
        on GPU if available
33 )
34 model.eval() # Set to inference mode (disables
    dropout, gradient tracking)
35
36 print(f"Model loaded. Parameters: {sum(p.numel() for p
    in model.parameters()):,}")
37 print(f"Device: {next(model.parameters()).device}")
38 print()
```

```

39
40 # -- Step 2: Define our inference function
41 -----
42 def run_inference(prompt: str, max_new_tokens: int =
43 100) -> dict:
44     """
45     Runs a single inference request and returns the
46     result with metrics.
47
48     Returns a dict containing:
49     - output_text: the generated text
50     - input_tokens: number of tokens in the prompt
51     - output_tokens: number of tokens generated
52     - ttft_ms: time to first token
53       (milliseconds)
54     - total_ms: total generation time
55       (milliseconds)
56     - tokens_per_sec: throughput (tokens/second)
57     """
58
59 # Step 2a: Tokenise the input
60 # Convert raw text \textrightarrow{} token IDs
61 \textrightarrow{} tensor on GPU
62 inputs = tokeniser(prompt,
63     return_tensors="pt").to(model.device)
64 input_len = inputs["input_ids"].shape[1] # number
65 of input tokens
66
67 # Step 2b: Record start time for TTFT measurement
68 t_start = time.perf_counter()
69
70 # Step 2c: Run inference
71 # torch.no_grad() tells PyTorch NOT to track
72 gradients

```

```
64     # (we don't need them for inference, and they would
        waste memory)
65     with torch.no_grad():
66         output_ids = model.generate(
67             **inputs,
68             max_new_tokens=max_new_tokens,
69             do_sample=False,          # Greedy decoding
            (deterministic)
70             pad_token_id=tokeniser.eos_token_id,
71         )
72
73     t_end = time.perf_counter()
74
75     # Step 2d: Compute metrics
76     output_len = output_ids.shape[1] - input_len    #
        tokens we generated
77     total_ms = (t_end - t_start) * 1000             #
        total time in ms
78     tokens_per_sec = output_len / (t_end - t_start)
79
80     # Step 2e: Decode the output back to text
81     generated_tokens = output_ids[0, input_len:]    #
        only new tokens
82     output_text = tokeniser.decode(generated_tokens,
        skip_special_tokens=True)
83
84     return {
85         "output_text":    output_text,
86         "input_tokens":   input_len,
87         "output_tokens":  output_len,
88         "total_ms":       total_ms,
89         "tokens_per_sec": tokens_per_sec,
90     }
91
```

```
92
93 # -- Step 3: Run some example requests
94     -----
95 prompts = [
96     "Explain what artificial intelligence is in simple
97     terms:",
98     "The key difference between training and inference
99     in machine learning is:",
100     "A GPU is better than a CPU for deep learning
101     because:",
102 ]
103
104 for i, prompt in enumerate(prompts, 1):
105     print(f"{'='*60}")
106     print(f"Request {i}: {prompt[:60]}...")
107     print()
108
109     result = run_inference(prompt, max_new_tokens=80)
110
111     print(f"    Input tokens: {result['input_tokens']}")
112     print(f"    Output tokens: {result['output_tokens']}")
113     print(f"    Total time: {result['total_ms']:.1f}
114           ms")
115     print(f"    Throughput:
116           {result['tokens_per_sec']:.1f} tokens/sec")
117     print()
118     print(f"    Generated text:")
119     print(f"    {result['output_text'][:200]}")
120     print()
121
122 # -- Step 4: Batch inference comparison
123     -----
124 print(f"{'='*60}")
125 print("Comparing single vs batch inference:")
```

```
119 print()
120
121 # Run requests one-by-one
122 t_sequential_start = time.perf_counter()
123 for prompt in prompts:
124     run_inference(prompt, max_new_tokens=50)
125 t_sequential = time.perf_counter() - t_sequential_start
126
127 print(f" Sequential (3 requests):
128       {t_sequential*1000:.0f} ms")
129 print()
130 print(" Note: In production, continuous batching
131       processes multiple")
132 print(" requests simultaneously, dramatically
133       improving throughput.")
134 print(" We will cover this in Chapter 4.")
```

Listing 1.1: Simple LLM inference with performance measurement

Running this script on an NVIDIA A100 GPU with GPT-2 produces output similar to:

```
Loading model: gpt2
Model loaded. Parameters: 117,222,400
Device: cuda:0

=====
Request 1: Explain what artificial intelligence is in
simpl...

Input tokens: 12
Output tokens: 80
Total time: 245.3 ms
Throughput: 326.1 tokens/sec

=====
Comparing single vs batch inference:
```

```
Sequential (3 requests): 712 ms
```

Listing 1.2: Sample output from basic inference script

What to Observe

Notice that 80 tokens took 245ms total. This is about 3ms per token. On a larger model (e.g. LLaMA-3-70B), you would see:

- Much higher memory usage (70B params $\times$ 2 bytes = 140GB)
- Slower throughput (perhaps 20–50 tokens/sec per request)
- Need for multi-GPU setup

These numbers motivate everything we will optimise in this book.

1.11 Performance Measurement: The Metrics That Matter

Before we end this chapter, we need to establish a shared vocabulary for measuring inference system performance. These metrics will appear throughout every chapter of this book.

1.11.1 Latency Metrics

Time to First Token (TTFT)

The time elapsed from when the request is received to when the *first output token* is generated.

Formula: $\text{TTFT} = t_{\text{first_token}} - t_{\text{request_received}}$

Typical targets: $< 200\text{ms}$ for interactive applications.

Time Per Output Token (TPOT)

The average time to generate each output token after the first one. This is the inverse of the per-request generation speed.

$$\text{Formula: TPOT} = \frac{t_{\text{last_token}} - t_{\text{first_token}}}{N_{\text{output_tokens}} - 1}$$

Typical targets: 20–50ms/token for single-stream, lower with batching.

End-to-End Latency

Total time from request submission to complete response received.

$$\text{Formula: E2E Latency} = \text{TTFT} + \text{TPOT} \times (N_{\text{output_tokens}} - 1)$$

Used for SLA (Service Level Agreement) commitments.

1.11.2 Throughput Metrics

Requests Per Second (RPS)

How many complete inference requests the system can handle per second across all concurrent users.

This is the primary *capacity* metric for inference systems.

Output Tokens Per Second (TPS)

Total output tokens generated per second across all requests.

$$\text{Formula: TPS} = \sum_{\text{requests}} \frac{N_{\text{output_tokens}}}{t_{\text{generation}}}$$

This is the primary metric for comparing inference engine efficiency.

1.11.3 Efficiency Metrics

GPU Utilisation

Percentage of time the GPU's compute units are actively performing calculations (as opposed to waiting for memory or network).

Target: > 70% in production systems. Below 50% suggests the system is bottlenecked by memory bandwidth or request scheduling.

Cost Per Million Tokens (\$/MTok)

The operational cost to generate one million output tokens. Includes GPU compute, memory, networking, and power costs.

This is the primary *business* metric for inference operations. Industry targets: \$0.10–\$5.00 per million tokens depending on model size and infrastructure.

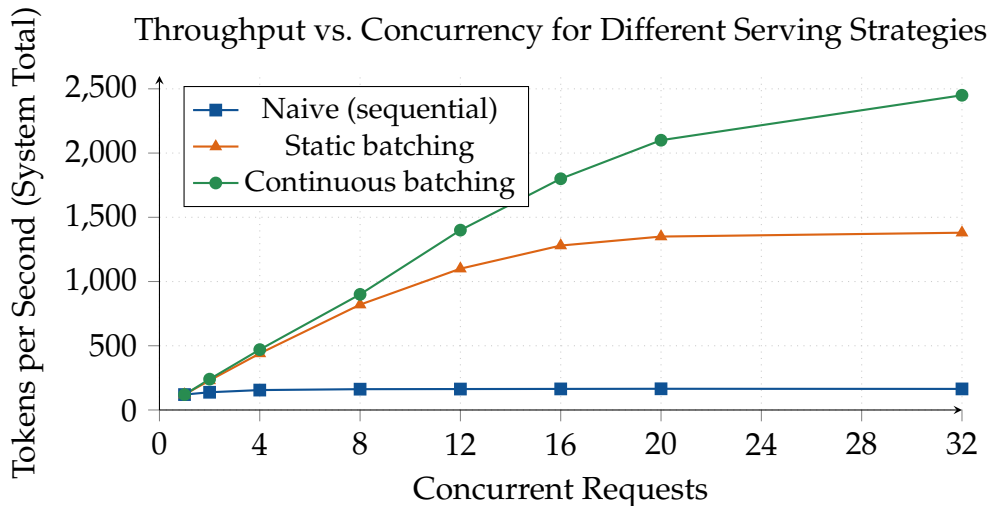


Figure 1.6: Throughput scaling with concurrent requests for three serving strategies. Continuous batching (vLLM-style) maintains near-linear scaling to much higher concurrency levels. We will explain exactly why in Chapter 4.

1.12 Production Engineering Notes

Before we close this chapter, here are some production realities that you will encounter as an inference engineer. These are lessons learned from running real systems at scale.

1.12.1 The Cold Start Problem

When a model is first loaded onto a GPU (or after a GPU is restarted), the first few requests are significantly slower because:

- Model weights must be transferred from disk/CPU to GPU memory
- CUDA kernels must be compiled and cached
- Memory allocators warm up

Production systems use **warm-up requests** — dummy requests sent at startup to initialise the system before real traffic arrives.

1.12.2 Memory Fragmentation

Even with sophisticated memory management, GPU memory can become fragmented over time — unable to allocate large contiguous blocks even when there is enough total free memory. Production systems monitor memory fragmentation and periodically restart workers to clean up.

1.12.3 Long-Tail Latency

The average latency might be excellent (200ms) but the P99 latency (worst 1% of requests) might be terrible (5 seconds). This happens due to:

- Garbage collection pauses
- Memory pressure causing evictions
- Network jitter
- Unlucky scheduling

Always monitor and design to P99, not just averages.

1.12.4 Model Versioning and Rollouts

Updating a model in production while maintaining availability requires careful blue-green deployment or canary rollouts. With large models (hundreds of GB), the update itself takes minutes — during which the new model must be loaded without dropping service.

1.13 Common Mistakes in Inference Engineering

Mistake 1: Confusing Training Metrics with Inference Metrics

A model with excellent training loss does not necessarily produce good inference performance. Training optimises model quality. Inference engineering optimises system performance. They are separate concerns.

Mistake 2: Ignoring Memory Bandwidth

Most inference engineers initially focus on compute (FLOPS). In practice, LLM inference is almost always *memory-bandwidth-bound*, not compute-bound. Optimising for the wrong bottleneck wastes engineering effort.

Mistake 3: Benchmarking with Unrealistic Inputs

Benchmarking with short prompts and single requests gives misleading results. Always benchmark with realistic input distributions, concurrency levels, and request patterns that match your actual production workload.

Mistake 4: Under-provisioning for Peak Load

AI inference traffic is highly spiky — product launches, viral moments, and business hours can cause $10\times$ normal traffic in minutes. Always provision for $2\text{--}3\times$ expected peak, and have clear shedding/queueing strategies for higher load.

1.14 Best Practices

Best Practice 1: Measure Before Optimising

Before optimising anything, measure your current system's bottleneck. Use profiling tools (NVIDIA Nsight, PyTorch Profiler, vLLM's built-in metrics) to determine whether you are compute-bound, memory-bound, or I/O-bound. Then optimise the actual bottleneck.

Best Practice 2: Use the Right Hardware

Not all GPUs are equal for inference.

- For memory-bandwidth-limited workloads (most LLMs): choose GPUs with highest HBM bandwidth (H100, A100)
- For compute-limited workloads (diffusion models, small models): choose GPUs with highest FLOPS
- For cost-sensitive deployments: consider A10G, L4 — cheaper with lower throughput, acceptable for low-concurrency workloads

Best Practice 3: Monitor the Full Stack

Monitor metrics at every layer:

- Application: TTFT, E2E latency, error rates
- Inference engine: batch sizes, queue depth, cache hit rate
- GPU: utilisation, memory usage, memory bandwidth
- Infrastructure: network latency, disk I/O

Slow P99 latency could be caused by a problem at *any* layer.

1.15 Chapter Summary

Chapter Summary

Key concepts from Chapter 1:

- **AI Inference** is the process of running a trained model on new data to produce outputs. The model parameters are fixed; only the input varies.
- **Training vs Inference** are fundamentally different: training changes parameters (slow, done once); inference applies fixed parameters (fast, done millions of times).
- **The inference request lifecycle** involves: network ingress, scheduling, tokenisation, prefill (parallel), decode (sequential), and streaming output.
- **The inference trilemma**: you must balance latency, throughput, and cost. Optimising all three simultaneously is the core challenge of the discipline.
- **Hardware matters**: GPUs dominate because neural networks are fundamentally massive parallel matrix operations. Memory bandwidth — not raw compute — is the primary bottleneck for most LLM inference.
- **Modern inference engines** (vLLM, TensorRT-LLM, TGI) provide production-grade features: continuous batching, KV caching, quantisation, and tensor parallelism.
- **Key metrics**: TTFT, TPOT, E2E latency, tokens per second, GPU utilisation, and \$/MTok.

In the next chapter, we will go deep on the transformer architecture — understanding exactly *what* the GPU is computing during inference and *why* the model is structured the way it is.

1.16 Exercises

Exercise 1: Measuring Inference Metrics

Run the code example from Listing 1.1 with GPT-2. Then modify it to:

1. Measure the time for just the tokenisation step vs the model forward pass.
2. Plot throughput (tokens/sec) as a function of output sequence length (try 10, 50, 100, 200, 500 tokens).
3. Calculate the cost estimate: assume the machine costs \$3/hour to run. What is your \$/MTok?

Exercise 2: Understanding the Memory Constraint

GPT-3 has 175 billion parameters.

1. If stored in FP32 (4 bytes per parameter), how many GB of memory does it require?
2. In FP16 (2 bytes per parameter)?
3. In INT8 (1 byte per parameter)?
4. An H100 GPU has 80GB of VRAM. How many H100s would you need to hold GPT-3 in FP16?
5. Research online: how many GPUs does OpenAI reportedly use to serve GPT-3/GPT-4? Does this match your calculation?

Exercise 3: The Trilemma in Practice

Consider three hypothetical AI serving systems, each optimising for a different corner of the latency–throughput–cost trilemma:

1. A real-time voice assistant (optimise for?)
2. A batch document processing pipeline running nightly (optimise for?)
3. A startup API offering cheap LLM access (optimise for?)

For each, identify which metric is *most* and *least* important, and describe one concrete engineering decision they would make differently.

Exercise 4: Architecture Exploration

Draw (on paper or digitally) the full request lifecycle diagram from Figure 1.3 from memory. Then:

1. Identify which stage you think takes the longest. Why?
2. Identify which stage is hardest to scale horizontally. Why?
3. If you needed to reduce TTFT by 50%, which stage would you focus on?

1.17 Further Reading

Further Reading & References

Foundational Papers

- Vaswani et al. (2017). *Attention Is All You Need*. The original transformer paper. Understanding this is prerequisites for Chapter 2.
- Kwon et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention*. The paper that introduced vLLM’s core innovation.
- Pope et al. (2022). *Efficiently Scaling Transformer Inference*. Google’s

analysis of LLM inference bottlenecks.

Engineering Resources

- vLLM documentation: <https://docs.vllm.ai>
- NVIDIA TensorRT-LLM: <https://github.com/NVIDIA/TensorRT-LLM>
- Hugging Face TGI: <https://github.com/huggingface/text-generation-inference>
- Andrej Karpathy’s “LLM in 100 lines” — builds intuition for the transformer forward pass.

Background Reading

- Dettmers et al. (2022). *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale*. Introduction to quantisation (Chapter 6 preview).
- Orca paper (Yu et al., 2022). *Orca: A Distributed Serving System for Transformer-Based Language Generation Models*. Introduced continuous batching.
- Leviathan et al. (2023). *Fast Inference from Transformers via Speculative Decoding*. A clever technique we will cover in Chapter 7.

Online Resources

- <https://modal.com/blog> — excellent practical articles on LLM serving
- <https://www.anyscale.com/blog> — Ray Serve and LLM infrastructure deep dives
- MLCommons MLPerf Inference benchmarks — <https://mlcommons.org/benchmarks/inference-datacenter/>

End of Chapter 1 · Next: Chapter 2 — The Transformer Architecture: What the GPU Is Actually Computing